

MYSQL WITH NODEJS REPORT

Question1: 🤔 But why pool.promise()?

By default, mysql2 gives you a **callback-based** API like:

js

CopyEdit

```
pool.query('SELECT * FROM users', (err, results) => {  
  // callback hell 🤯  
});
```

But when you use:

js

CopyEdit

```
const db = pool.promise();
```

You unlock the ability to use **async/await**, like:

js

CopyEdit

```
const [rows] = await db.execute('SELECT * FROM users WHERE email = ?', [email]);
```

This is **cleaner, modern, and more readable** — especially when you're building apps with multiple DB calls.

Summary

Without .promise()

Uses callback

Less readable in large code

Older-style API

With .promise()

Uses async/await

Cleaner and easier to manage

Modern and preferred

Yes — **SQLite (using sqlite3 module)** is **callback-based by default**, but you can also use `async/await` if you use a different library or wrap the functions yourself.

Option 1: Use sqlite3 with Promisified Functions (Manual)

You can use Node.js's built-in `util.promisify` to convert callbacks to promises:

js

CopyEdit

```
const sqlite3 = require('sqlite3').verbose();
```

```
const { promisify } = require('util');
```

```
const db = new sqlite3.Database('./database.db');
```

```
// Promisify db methods
```

```
const runAsync = promisify(db.run.bind(db));
```

```
const getAsync = promisify(db.get.bind(db));
```

```
const allAsync = promisify(db.all.bind(db));
```

```
// Example usage
```

```
(async () => {
```

```
  const users = await allAsync('SELECT * FROM users');
```

```
  console.log(users);
```

```
}());
```

Option 2: Use sqlite (Better for async/await)

Instead of `sqlite3`, you can use the `sqlite` or `better-sqlite3` package, which supports Promises out of the box.

Install it:

bash

CopyEdit

```
npm install sqlite sqlite3
```

Sample usage:

js

CopyEdit

```
const sqlite3 = require('sqlite3');
```

```
const { open } = require('sqlite');
```

```
async function setup() {
```

```
  const db = await open({
```

```
    filename: './database.db',
```

```
    driver: sqlite3.Database
```

```
  });
```

```
  const users = await db.all('SELECT * FROM users');
```

```
  console.log(users);
```

```
}
```

```
setup();
```

Summary

Approach	Callback Promise Support Notes		
sqlite3	✓	✗ (by default)	Use util.promisify for async/await
sqlite + sqlite3	✓	✓	Best if you want clean async/await
better-sqlite3	✗	✓	Synchronous API, but fast & elegant

✓ Q1: Why is the table not created like in SQLite?

In **SQLite**, when you connect to the DB and call `db.run('CREATE TABLE IF NOT EXISTS...')`, it immediately creates the table.

But in **MySQL**, tables are not auto-created unless you explicitly run a CREATE TABLE query.

 **Solution:** You must run a setup/init script once (e.g., during server startup or via a migration tool) to create your tables.

js

CopyEdit

```
// setup.js
```

```
const db = require('./config/db');
```

```
(async () => {
```

```
  await db.execute(`
```

```
    CREATE TABLE IF NOT EXISTS users (
```

```
      id INT AUTO_INCREMENT PRIMARY KEY,
```

```
      email VARCHAR(255) UNIQUE,
```

```
      password VARCHAR(255)
```

```
    )
```

```
`);
```

```
  console.log('Users table ready');
```

```
})();
```

✅ Q2: Why are receiving variables wrapped with [], like: [result]?

This is because of how mysql2's .execute() method works when using promise().

js

CopyEdit

```
const [rows, fields] = await db.execute(query, values);
```

- It returns an **array with two elements**:
 - rows: actual result (e.g., data from SELECT)
 - fields: metadata about columns (often unused)

So when you do:

js

CopyEdit

```
const [result] = await db.execute(...);
```

You're destructuring the first item (rows or result) from the response array.

✓ Clean and readable, but yes — [result] is because execute() returns [rows, fields].

✓ Q3: If promise is rejected, what will be returned to the caller?

If an error occurs in this function:

js

CopyEdit

```
const createUser = async (email, password) => {  
  const [result] = await db.execute(...);  
  return result.insertId;  
};
```

And db.execute(...) fails, it **throws an error**.

So what happens?

- The function doesn't return anything.
- It **throws**, and the error **bubbles up** to the caller.
- You **must catch** it using either:
 - try...catch
 - or .catch() if using .then()

📄 Example:

js

CopyEdit

```
try {  
  const id = await createUser("satish@example.com", "pass123");  
  console.log("User created with id:", id);  
} catch (err) {  
  console.error("Error creating user:", err.message);  
}
```

🔍 Can we write table creation code inside the model file like SQLite?

Yes, **you absolutely can**, but with a small caveat:

Unlike SQLite (which is embedded and lightweight), **MySQL is a server-based DB**, so best practice is to **run table creation once** (e.g., at app start or via migrations). But technically, you **can write creation logic inside each model** just like you do in SQLite.

✅ **Example: MySQL Table Creation Inside Model File**

js

CopyEdit

// models/User.js

```
const db = require('../config/db');
```

```
const initUserTable = async () => {
```

```
  await db.execute(`
```

```
    CREATE TABLE IF NOT EXISTS users (
```

```
      id INT AUTO_INCREMENT PRIMARY KEY,
```

```
      email VARCHAR(255) UNIQUE,
```

```
      password VARCHAR(255)
```

```
    )
```

```
  `);
```

```
};
```

```
const createUser = async (email, password) => {
```

```
  const [result] = await db.execute(
```

```
    'INSERT INTO users (email, password) VALUES (?, ?)',
```

```
    [email, password]
```

```
  );
```

```
  return result.insertId;
```

```
};
```

```
const findUserByEmail = async (email) => {
```

```
  const [rows] = await db.execute(
```

```
    'SELECT * FROM users WHERE email = ?',
```

```
    [email]
```

```
);  
return rows[0];  
};  
  
module.exports = {  
  initUserTable,  
  createUser,  
  findUserByEmail,  
};
```

Use `initUserTable()` During App Startup

```
js  
CopyEdit  
// app.js or index.js  
const express = require('express');  
const userModel = require('./models/User');  
  
(async () => {  
  await userModel.initUserTable(); // create table if not exists  
})();  
  
const app = express();  
// your other middlewares and routes here
```

Pro Tip

-  **Good for small projects or prototyping**
 -  For larger apps, prefer **migrations** (like `sequelize-cli` or [knex migrations](#)) — avoids conflicts and improves version control of DB schemas.
-
-

🤔 Why Use Migrations in Large Apps (like sequelize-cli, knex, etc.)?

Migrations = **version-controlled scripts that modify your DB schema** (e.g., create/alter tables). They act like **Git for your database**.

🧠 Without Migrations

Suppose you write CREATE TABLE in your model files. You run the app, table is created.

But what happens when:

1. You **change the schema** later (e.g., add a new column)?
2. Someone else on your team also changes the schema differently?
3. You have to deploy to staging, prod, etc.?

You'd end up with:

- Chaos
 - Conflicting schemas
 - No history or rollback
 - Manual syncing 🚫
-

🚀 With Migrations

Think of migrations like this:

- ✅ Every schema change is a **separate versioned JS/SQL file**
 - ✅ You commit it to Git
 - ✅ You can run:
 - migrate up → applies new changes
 - migrate down → rolls back last change
-

📦 Example (Using Knex)

bash

CopyEdit

```
npx knex migrate:make create_users_table
```

Generates a migration file like:

js

CopyEdit

```
exports.up = function(knex) {  
  return knex.schema.createTable('users', (table) => {  
    table.increments('id');  
    table.string('email').unique();  
    table.string('password');  
  });  
};
```

```
exports.down = function(knex) {  
  return knex.schema.dropTable('users');  
};
```

Now you run:

bash

CopyEdit

```
npx knex migrate:latest
```

→ Table created.

And later, if you mess up or want to revert:

bash

CopyEdit

```
npx knex migrate:rollback
```

→ Table dropped or last change undone.

✅ Benefits of Migrations

Feature	Why It's Useful
 Version Control	Every change is tracked + stored in Git
 Team-Friendly	Everyone can sync DB schema consistently
 CI/CD Ready	Can auto-run migrations in deployment
 Rollbacks	You can undo a bad migration easily

Feature	Why It's Useful
---------	-----------------

 Environment Sync	Dev, test, and prod DBs always stay in sync
--	---

Tools

- **Knex:** Great for raw SQL control
 - **Sequelize:** ORM with its own migration system
 - **Prisma:** Uses prisma migrate (schema-first)
-
-

✅ **mysql -u root -p < schema.sql**

This command **executes the SQL commands inside schema.sql** file using the MySQL CLI.

Breakdown:

Part	Meaning
mysql	Invokes the MySQL command-line tool
-u root	Logs in with the username root (you can change it to your MySQL user)
-p	Prompts for the password (it's secure—you type it, but it won't show)
< schema.sql	Redirects the contents of schema.sql file into the MySQL CLI

What happens:

It's the same as opening the MySQL shell and typing everything inside schema.sql line-by-line. This way:

- The **database (node_app) is created** (if not exists).
 - The **tables (users, blogs) are created**.
-

Pro Tip:

If your username isn't root, replace it like:

bash

CopyEdit

```
mysql -u yourUsername -p < schema.sql
```

And if MySQL is on another host:

```
bash
```

CopyEdit

```
mysql -h yourhost.com -u root -p < schema.sql
```